EXPERIMENT NO. 10 · MINI PROJECT

# CineGraph

Movie Recommendation System Using Neo4j Graph Database — *"Discover Movies Through Connections"*

**What is CineGraph?**

A full-stack web application that recommends movies by following the **connections** between users, movies, genres, actors and directors stored in a **Neo4j graph database**.

- Browse and search 80 movies with genre, year and rating filters
- Like, rate (1–5★) and mark movies as watched
- Personal recommendations with a match % and a clear reason
- Interactive graph explorer showing how everything is connected
- Insights page with statistics calculated from the graph



| Group No. 3 · Final Year B.Tech · Div. A | | |
|---|---|---|
| **Unique ID** | **Roll No.** | **Name** |
| EN23192312 | 11 | Shreeya Amit Kalebere |
| EN23120981 | 12 | Shubham Shankar Pattewar |
| EN23217963 | 13 | Anuj Arun Powar |
| EN23250682 | 14 | Virendra Vilas Katale |
| EN23267649 | 15 | Preeti Nishikant Sonule |



**Built with:** React · Vite · Tailwind CSS · Cytoscape.js · Node.js · Express · Neo4j · Cypher

# The problem, and why a graph database

Movie data is full of connections. A good recommender must follow them quickly — this is exactly what a graph database is built for.

### Problem statement

- Users waste time deciding what to watch.
- "Top rated" lists show the **same movies to everyone**.
- Most apps **never explain** why a movie is suggested.
- In SQL, following links (user → movie → director → other movies) needs **many JOINs**.

### Our aim

Store movies, people and user activity as a **graph in Neo4j**, recommend movies by walking the graph with **Cypher**, and show the **reason** behind every suggestion.

```
// Neo4j: one pattern finds movies by the same
// director as a movie the user liked
MATCH (u:User)-[:LIKED]->(m:Movie)
      <-[:DIRECTED]-(d:Director)-[:DIRECTED]->(rec:Movie)
```

### Types of NoSQL databases

| Type | Stores data as | Example |
|---|---|---|
| Key–value | Key and value | Redis |
| Document | JSON-like documents | MongoDB |
| Column-family | Rows with flexible columns | Cassandra |
| **Graph** | **Nodes and relationships** | **Neo4j (our choice)** |

### SQL database vs Neo4j

| Point | SQL database | Neo4j |
|---|---|---|
| Links between data | Foreign keys + join tables | Stored directly as relationships |
| "Same director as a movie I liked" | Several JOINs | One short pattern |
| Adding new kinds of data | Change table structure | Add labels / relationships any time |
| Showing data as a graph | Extra work | Result is already a graph |

# System architecture (three tiers)

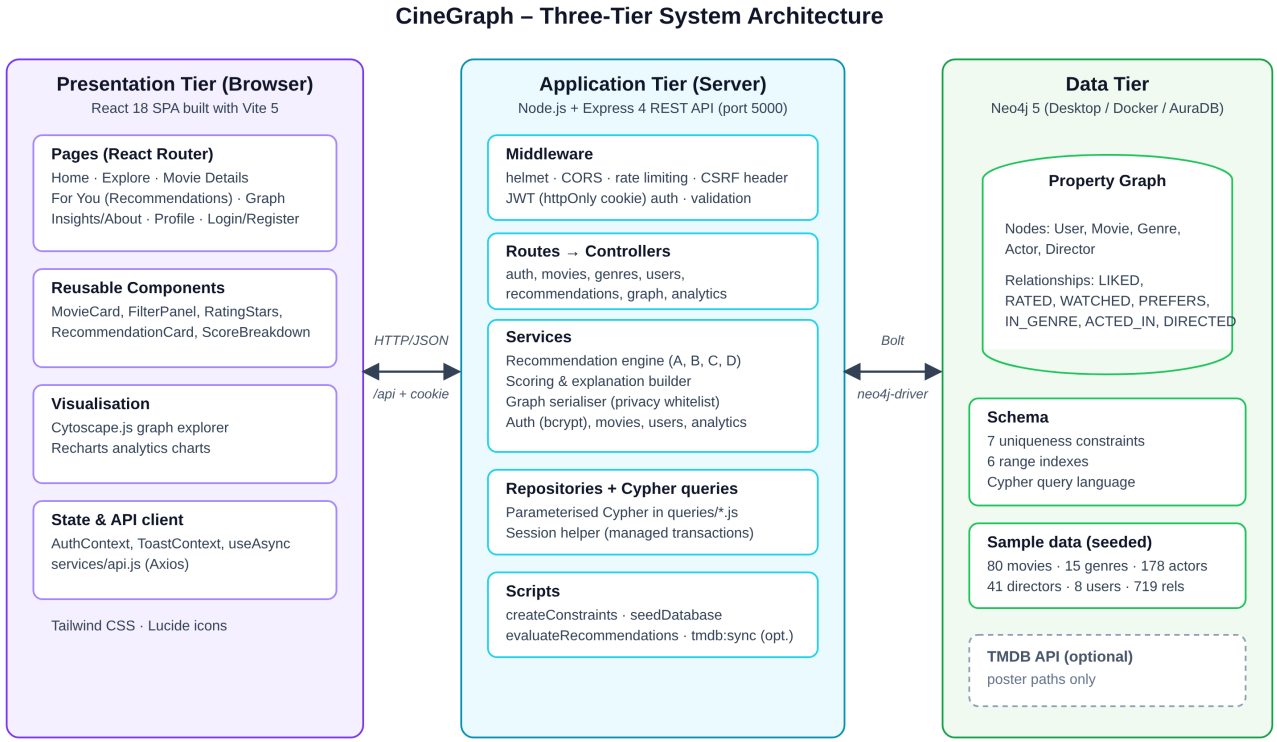



Figure 1 – Three-tier architecture of CineGraph

### 1. Presentation tier – Browser

React website built with Vite. Pages: Home, Explore, Movie details, For You, Graph, Insights, Profile, Login. Sends requests to the API and shows results.

### 2. Application tier – Server

Node.js + Express REST API. Checks login, validates input, runs the recommendation engine and builds the explanations.

### 3. Data tier – Neo4j

Stores every node and relationship. The API talks to it with the official Neo4j JavaScript driver over the **Bolt** protocol (port 7687) using Cypher.

# Technologies, data flow and security

## Technology stack

| Part | Technology |
|---|---|
| Frontend | React 18, Vite 5, Tailwind CSS, React Router, Axios |
| Graph view & charts | Cytoscape.js, Recharts |
| Backend | Node.js, Express 4 (REST API) |
| Database | Neo4j 5 (tested on 5.26) with Cypher |
| Database driver | Official `neo4j-driver` (Bolt protocol) |
| Security | bcryptjs, JWT, helmet, express-validator, rate limiting |
| Testing | Vitest, Supertest |

## Backend structure (layers)

**routes** → **validators** → **middleware** (auth) → **controllers** → **services** (recommendation engine) → **repositories** → **Cypher queries** → Neo4j

## How one request flows

1. User clicks something on the website.
2. React sends an HTTP request to the Express API.
3. The API checks the login cookie and validates the input.
4. The API runs a parameterised Cypher query on Neo4j.
5. Neo4j returns the matching nodes and relationships.
6. The API sends JSON back and the page shows it.

### Security features

- Passwords stored only as **bcrypt hashes**.
- Login session in a **JWT httpOnly cookie** (JavaScript cannot read it).
- A user can only change **their own** data; others get HTTP 403.
- All inputs validated; all Cypher uses **parameters** (no injection).
- Login attempts are **rate-limited**.

# Neo4j graph data model

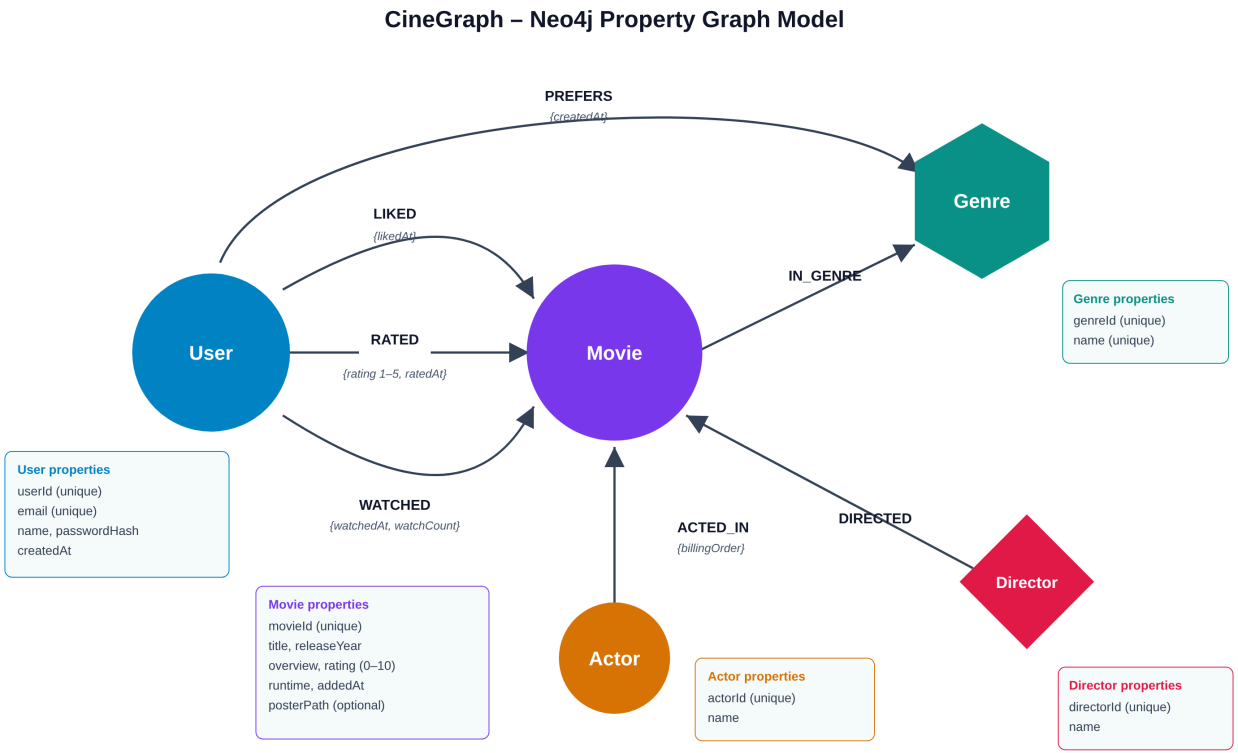



Figure 2 – Nodes, relationships and properties in CineGraph

### Property graph – 3 building blocks

- **Node** – a thing (User, Movie…). Its type is a **label**.
- **Relationship** – a named, directed link, e.g. `LIKED`.
- **Property** – a value on a node or link, e.g. `title`, `rating`.

| Node label | Key properties | Count |
|---|---|---|
| Movie | movieId, title, releaseYear, rating, runtime | 80 |
| Actor | actorId, name | 178 |
| Director | directorId, name | 41 |
| Genre | genreId, name | 15 |
| User | userId, name, email, passwordHash | 8 |
| **Total nodes** | | **322** |

# Relationships, constraints and Cypher

## Relationship types (719 in total)

| Relationship | Properties | Count |
|---|---|---|
| (User)-[:LIKED]→(Movie) | likedAt | 38 |
| (User)-[:RATED]→(Movie) | rating 1–5, ratedAt | 26 |
| (User)-[:WATCHED]→(Movie) | watchedAt, watchCount | 62 |
| (User)-[:PREFERS]→(Genre) | createdAt | 18 |
| (Movie)-[:IN_GENRE]→(Genre) | – | 220 |
| (Actor)-[:ACTED_IN]→(Movie) | billingOrder | 268 |
| (Director)-[:DIRECTED]→(Movie) | – | 87 |

**Indexes (6):** Movie(title), Movie(releaseYear), Movie(rating), Movie(addedAt), Actor(name), Director(name) – used for filtering and sorting.

## Uniqueness constraints (7) – no duplicate nodes

```
CREATE CONSTRAINT movie_id_unique IF NOT EXISTS
FOR (m:Movie) REQUIRE m.movieId IS UNIQUE;
// also: userId, email, genreId, genre name, actorId, directorId
```

## Saving a like – MERGE (no duplicate links)

```
MATCH (u:User {userId: $userId}), (m:Movie {movieId: $movieId})
MERGE (u)-[r:LIKED]->(m)
ON CREATE SET r.likedAt = datetime()
RETURN m.title AS liked
```

## Saving a rating – updates if rated again

```
MATCH (u:User {userId: $userId}), (m:Movie {movieId: $movieId})
MERGE (u)-[r:RATED]->(m)
SET r.rating = $rating, r.ratedAt = datetime()
```

**CREATE** always adds new data; **MERGE** adds it only if it is not already there. `$userId` etc. are **parameters**, which prevents Cypher injection.

# Recommendation algorithms – walking the graph

Each method starts from the movies the user liked (or rated 4★–5★) and follows a different path through the graph. Movies the user already liked, rated or watched are never suggested.

## A Genre-based

Liked movie → Genre → other movies in the same genres.

```
MATCH (u:User {userId: $userId})
  -[:LIKED]->(:Movie)
  -[:IN_GENRE]->(g:Genre)
  <-[:IN_GENRE]-(rec:Movie)
WHERE NOT EXISTS {
  (u)-[:LIKED|RATED|WATCHED]->(rec) }
RETURN rec.title AS movie,
  rec.rating AS rating,
  count(DISTINCT g) AS sharedGenres
ORDER BY sharedGenres DESC,
  rating DESC LIMIT 5
```

## B Actor / Director

Liked movie ← Actor → other movies with the same actors.

```
MATCH (u:User {userId: $userId})
  -[:LIKED]->(:Movie)
  <-[:ACTED_IN]-(a:Actor)
  -[:ACTED_IN]->(rec:Movie)
WHERE NOT EXISTS {
  (u)-[:LIKED|RATED|WATCHED]->(rec) }
RETURN rec.title AS movie,
  collect(DISTINCT a.name) AS sharedActors
ORDER BY size(sharedActors) DESC,
  movie ASC LIMIT 5
```

The director version uses `DIRECTED` instead of `ACTED_IN`.

## C Similar users

Liked movie ← other user → movies that user also liked.

```
MATCH (u:User {userId: $userId})
  -[:LIKED]->(:Movie)
  <-[:LIKED]-(other:User)
  -[:LIKED]->(rec:Movie)
WHERE other <> u AND NOT EXISTS {
  (u)-[:LIKED|RATED|WATCHED]->(rec) }
RETURN rec.title AS movie,
  rec.rating AS rating,
  count(DISTINCT other) AS similarUsers
ORDER BY similarUsers DESC,
  rating DESC LIMIT 5
```

Similarity = shared liked movies ÷ all movies of both users (**Jaccard index**).

These are simplified forms of the project's queries (all tested on the project database). The full versions also use 4★–5★ ratings, preferred genres and weights.

# Hybrid score, new users and explanations

## D. Hybrid match score

**Match = 30% Genre + 15% Actor + 15% Director + 25% Similar users + 15% Movie rating**

Each part is a number between 0 and 1. If a part cannot be calculated (for example, no similar users), it is left out and the other weights are scaled up, so the score is still out of 100%.

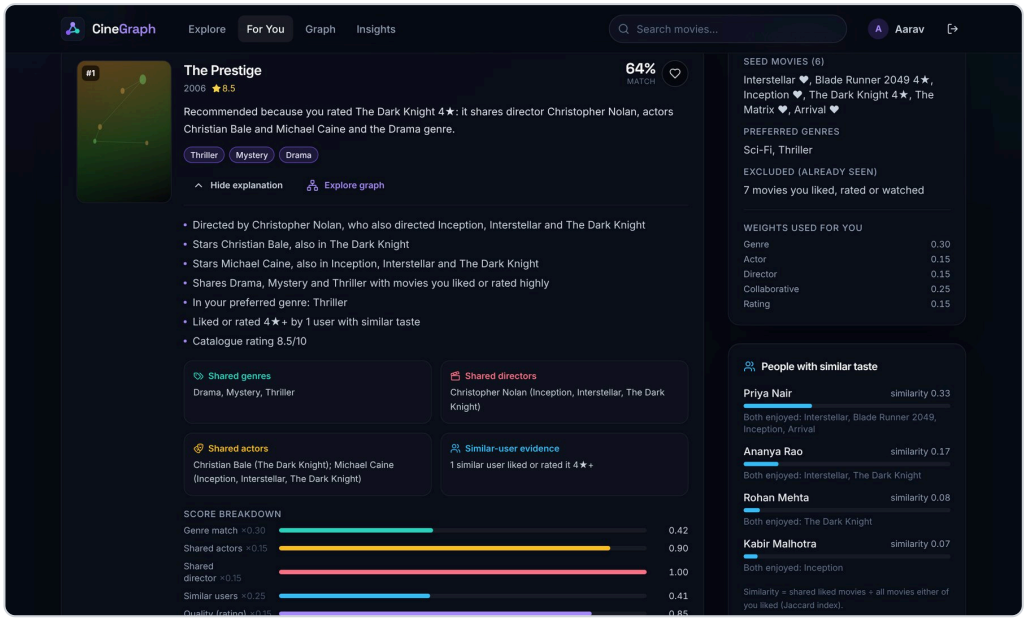
## New users (cold start)

| User has... | CineGraph shows |
|---|---|
| Nothing yet | Popular movies, marked “not personalised” |
| Only favourite genres | Best movies from those genres |
| At least one like / good rating | Fully personal recommendations |

## Explanations are always true

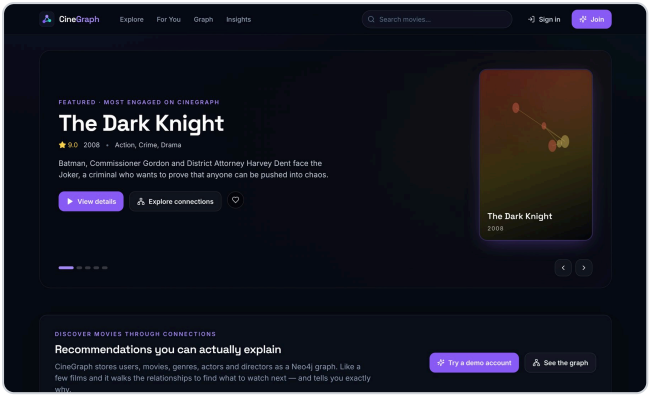
The reason is built only from what the Cypher query actually found – shared genres, shared actors/directors and similar users. Tests re-check these links in Neo4j.

## Real example – demo user Aarav, #1 recommendation

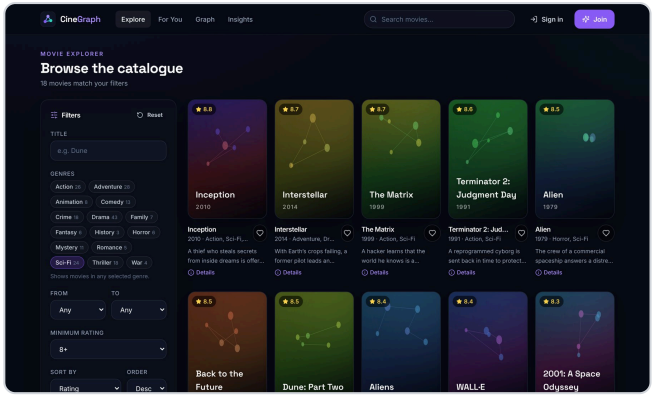



| Part | Score | Weight | Points |
|---|---|---|---|
| Genre | 0.42 | 0.30 | 0.125 |
| Actor (Bale, Caine) | 0.90 | 0.15 | 0.135 |
| Director (Nolan) | 1.00 | 0.15 | 0.150 |
| Similar users | 0.41 | 0.25 | 0.103 |
| Rating (8.5/10) | 0.85 | 0.15 | 0.128 |
| **The Prestige – match** | | | **0.64 = 64%** |

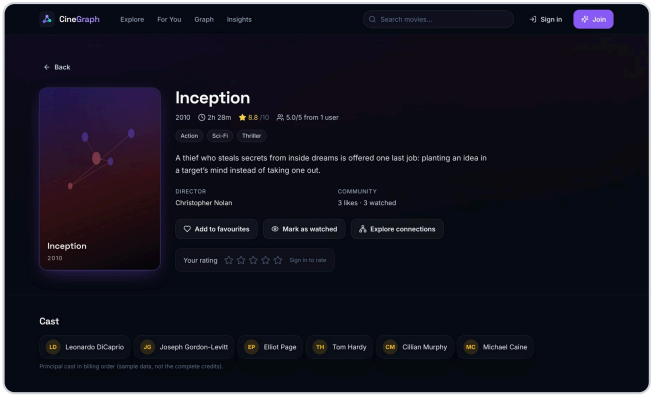
# The application – main screens



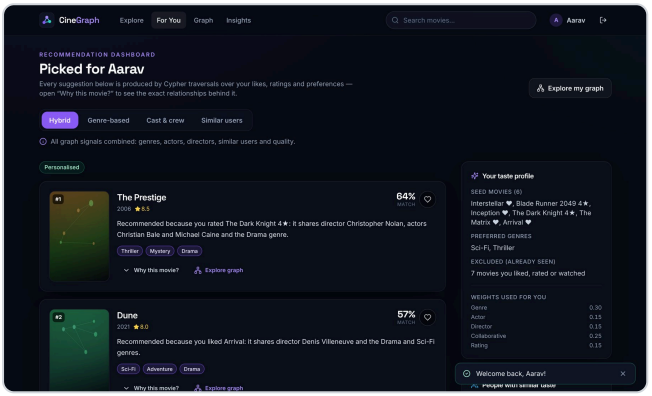

Home – featured and trending movies



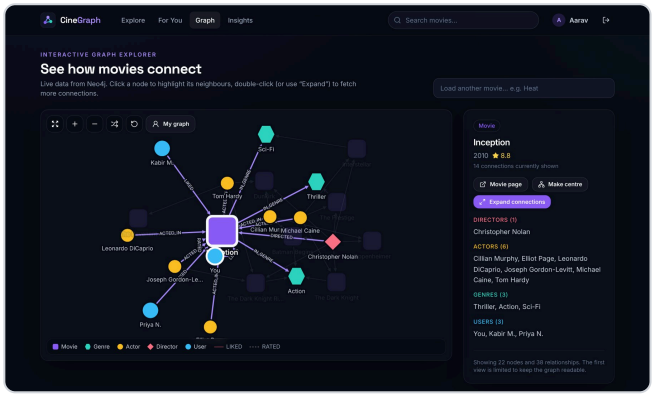

Explore – search, genre / year / rating filters



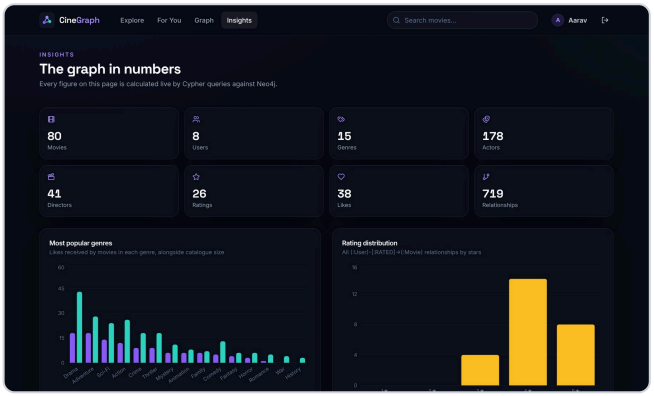

Movie details – like, rate, watch




For You – recommendations with match %




Graph explorer (Cytoscape.js) – live Neo4j data




Insights – statistics from Cypher queries

Graph explorer colours: **Movie**, **Genre**, **Actor**, **Director**, **User**. Clicking a node highlights its links; double-clicking loads more connections from Neo4j.

# Testing, results and conclusion

| 322 | 719 | 53 | 25 | 0 |
|---|---|---|---|---|
| nodes in the graph | relationships | unit / API tests passed | integration tests passed | duplicates when re-seeding |

## What was tested

- Neo4j connection, data loading and no duplicates on re-run
- Search, genre / year / rating filters and pagination
- Like, unlike, rating (only 1–5 allowed) and watch history
- All four recommendation methods; seen movies are excluded
- Explanations: every named actor / director really links the movies
- Wrong input → 400, not logged in → 401, other user's data → 403, database down → 503

## Limitations

- Small sample: 80 movies and 8 demo users.
- Score weights are chosen by hand, not learned.

## Future scope

- Larger movie dataset and real posters
- Neo4j Graph Data Science for faster similarity at scale
- Reviews, watch-lists and friends
- Mobile app using the same API

### Conclusion

CineGraph shows that a graph database is a natural fit for recommendations. Questions like "movies with the same director as one I liked" become one short Cypher pattern, and the same relationships power the recommendations, their explanations and the visual graph explorer.